



Universidad
de Alcalá

Escuela
Politécnica
Superior

La consola Linux

Sesiones S02 (1/2) · S03 (2/2)

Universidad de Alcalá · EPS · 780007 · Grupo 2E

Estructura de la sesión

- **S02 (1/2)** — fundamentos de la shell, navegación, ficheros, permisos, edición (nano + vim)
- **S03 (2/2)** — tipos de orden, control de trabajos, monitorización de procesos, redirecciones de E/S, pipes, scripting

Instalar Docker

Lo instalaremos juntos en clase — esta slide es tu referencia.

- **macOS:** instalar **Docker Desktop** desde <https://www.docker.com/products/docker-desktop/> (`.dmg`), y abrir la app Docker Desktop una vez
- **Windows:** mismo instalador de Docker Desktop — requiere **WSL2** activado (el propio instalador se ofrece a configurarlo si no lo está)
- **Linux:** instalar con el gestor de paquetes de tu distro (p.ej. `sudo apt install docker.io` en Ubuntu/Debian), y añadirte al grupo `docker` para no necesitar `sudo` cada vez: `sudo usermod -aG docker $USER` (cerrar sesión y volver a entrar para que tenga efecto)

Verificar que funciona, en cualquier sistema operativo:

```
docker --version
docker run hello-world
```

Arrancar el entorno de laboratorio

```
docker run -it \  
  -v $(pwd):/workspace \  
  -w /workspace \  
  opobla/sog-lab:2026-2027
```

- `-v $(pwd):/workspace` — monta tu directorio actual dentro del contenedor, para que los ficheros sobrevivan cuando el contenedor termine
- `-w /workspace` — empieza justo ahí, no en un directorio por defecto sin relación
- Misma imagen, mismo patrón (`make shell`) que Lab3/Lab4/Lab5 — lo volverás a ver

Qué es una shell, y por qué importa

- Un programa que lee lo que tecleas, lo interpreta como orden y le pide al SO que la ejecute
- `bash` (**B**ourne **A**gain **S**hell) — la shell por defecto en casi cualquier Linux
- Una breve línea de tiempo: Thompson shell (1971) → Bourne shell (1979) → bash
- Por qué sigue importando con interfaz gráfica everywhere: **control preciso + automatización** — y es la interfaz de casi cualquier servidor/contenedor/máquina en la nube que ejecuta software real

El prompt y los primeros comandos

El prompt muestra usuario, máquina, directorio actual y nivel de privilegio (`$` vs `#`).

- `date` , `cal` — fecha/hora actual, calendario
- `whoami` , `uname` — quién eres, qué sistema es este
- `echo` — imprimir texto
- `type` — ¿es una orden interna del shell o un ejecutable externo?
- `clear` , `exit`

```
sog@sog-lab:/workspace$ whoami
sog
sog@sog-lab:/workspace$ date
lun sep 14 12:03:11 UTC 2026
```

Historial, completado y ayuda

- `history` — todo lo tecleado en esta sesión; ↑/↓ para navegarlo
- `TAB` — completado (ficheros, órdenes — pulsa, no lo teclees entero)
- `man <orden>` — el manual, organizado en secciones:
 - 1 → órdenes generales, 2 → llamadas al sistema, 3 → funciones de biblioteca C

```
$ man ls
LS(1)                                User Commands                                LS(1)
NAME
    ls - list directory contents
SYNOPSIS
    ls [OPTION]... [FILE]...
```

`q` para salir de `man`, `/patrón` para buscar dentro.

El árbol de directorios

```
/
├── bin      (ejecutables)
├── etc      (configuración)
├── usr
├── mnt      (puntos de montaje)
├── home     (directorios de usuario)
├── dev      (dispositivos, como ficheros)
└── proc     (info del kernel, como ficheros)
```

Todo cuelga de una única raíz `/` — no hay letras de unidad.

Rutas absolutas y relativas

- **Absoluta:** empieza en `/`, no depende de dónde estés
- **Relativa:** parte de tu directorio actual
- `.` directorio actual · `..` directorio padre · `~` tu home

Rutas absolutas y relativas — un ejemplo trabajado

Partiendo de `/usr/local/bin`, las dos llegan a `/usr/local/share`:

```
$ cd /usr/local/share # absoluta: siempre funciona, sea cual sea el punto de partida
$ cd ../share         # relativa: sube un nivel a /usr/local, y baja a share
```

Mismo destino — pero la relativa solo funciona **porque** partías de `/usr/local/bin`. Tecléala desde otro sitio y te lleva a un destino completamente distinto.

Navegar el sistema de ficheros

- `pwd` — ¿dónde estoy?
- `ls` — ¿qué hay aquí? (`-a` ficheros ocultos, `-l` vista detallada)
- `cd` — cambiar de directorio

```
$ pwd
/workspace
$ ls
notes.txt  scripts
$ cd scripts
$ pwd
/workspace/scripts
```

Leer `ls -l`

```
drwxr-xr-x  2 root    root      4096 sep 12 08:42 subversion
-rw-rw-r--  1 parraman parraman  462 nov 30 12:52 instr.txt
```

permisos · enlaces · propietario · grupo · tamaño · última modificación · nombre

Manipular ficheros y directorios

- `cp` (`-r` para directorios), `mv` (mover/renombrar)
- `mkdir`, `rmdir` (solo directorios vacíos)
- `rm` (`-r` recursivo, `-i` confirma, `-f` fuerza — `rm -rf` sin pensarlo dos veces es como se borra lo que no querías borrar)

```
$ mkdir backup
$ cp notes.txt backup/
$ mv backup/notes.txt backup/notes-antiguo.txt
$ rm -i backup/notes-antiguo.txt
rm: ¿borrar el fichero regular 'backup/notes-antiguo.txt'? y
```

Comodines y variables

- `*` casa cualquier secuencia de caracteres, `?` casa exactamente uno
- Variables de shell (solo esta sesión) vs. de entorno (heredadas por los programas que lances):
`$NOMBRE` para usarlas — `HOME`, `SHELL`, `PATH` ya vienen definidas

```
$ ls *.txt           # cualquier fichero que acabe en .txt
notes.txt todo.txt
$ echo $HOME
/home/sog
```

Permisos

- Tres clases (lectura, escritura, ejecución) × tres sujetos (propietario, grupo, resto)
- `chmod` los cambia — simbólico (`chmod u+x fichero`) u octal (`chmod 755 fichero` , r=4/w=2/x=1 por sujeto)

Permisos — propietario, grupos y `/etc/passwd`

- El **propietario** es la cuenta de usuario que creó el fichero (o a quien se le asignó con `chown`) — la misma cuenta con la que se hace login (`whoami`)
- Cada usuario pertenece a uno o más **grupos** — un conjunto de usuarios que comparten cierto permiso (p.ej. un grupo `docencia` con escritura sobre una carpeta compartida, sin dársela a cualquiera)
- Esa información vive en `/etc/passwd` (una línea por usuario: nombre, UID, GID principal, home, shell) y en `/etc/group`

```
-rw-rw-r-- 1 parraman parraman 462 nov 30 12:52 instr.txt
           ↑ propietario ↑ grupo
```

Propietario y grupo (ambos `parraman` aquí) pueden leer+escribir; el resto solo lee.

Editar con nano

- Curva de entrada cero: las teclas que necesitas están en pantalla
- `^O` guardar · `^X` salir · `^W` buscar
- Suficiente para editar cualquier cosa desde el minuto uno

Editar con vim

vim funciona en **modos**, no en escritura libre:

- **Modo normal:** moverse, dar órdenes (`dd` borra línea, `yy` copia, `/patrón` busca)
- **Modo inserción** (`i` para entrar): escribir texto normal
- **Modo línea de comandos:** `:w` guardar, `:q` salir, `:wq` ambos, `:q!` salir sin guardar

Ejecuta `vimtutor` (ya instalado) para una práctica guiada de 25-30 min.

El papel de `Esc` : siempre te devuelve a modo normal, sea cual sea el modo en el que estés — la forma fiable de "salir" de donde estés antes de dar la siguiente orden.

```
Esc → i (escribe tu texto) → Esc → :wq
```

Teclear `:wq` estando todavía en modo inserción no guarda ni sale — simplemente escribe `:wq` literalmente en el texto. Pulsa siempre `Esc` primero.

Checkpoint: fin de S02 (1/2)

Tipos de orden

- **Ejecutable** (fichero en disco, localizable con `which`)
- **Interna del shell** (`cd` , `echo` , ...)
- **Script** (fichero de texto con órdenes, ejecutado por un intérprete)
- **Alias** (nombre corto para otra orden)

`type <orden>` dice cuál es cuál.

```
$ type cd
cd is a shell builtin
$ type ls
ls is /usr/bin/ls
$ which ls
/usr/bin/ls
```

Control de trabajos

- Las órdenes se ejecutan en **primer plano** (bloquea el prompt) o en **segundo plano** (terminando la línea con `&`)
- `jobs` — lista los trabajos en segundo plano
- `Ctrl+Z` — suspende el trabajo en primer plano
- `fg` / `bg` — lo reanuda en primer/segundo plano

```
$ sleep 100 &
[1] 4821
$ jobs
[1]+  Running      sleep 100 &
$ fg
sleep 100          # ahora bloquea el prompt de nuevo
```

Monitorizar procesos

- `ps` — lista los procesos vivos
- `top` — vista continua en vivo
- `kill` — envía una señal a un proceso (por defecto, `SIGTERM` , "termina por favor")

```
$ ps aux | grep sleep
sog  4821  0.0  0.0   2384  1536 pts/0  S   12:04   0:00 sleep 100
$ kill 4821
```

Esto es solo el uso básico. Qué es realmente una señal, o cómo decide el planificador a quién dar CPU, es `05-processes-threads` / `07-processes-lab` — llega pronto.

Entrada/salida estándar y redirecciones

Todo proceso nace con tres canales: 0 = entrada estándar, 1 = salida estándar, 2 = salida de error (por defecto, todos el terminal).

- `<` redirige la entrada desde un fichero
- `>` / `>>` redirige la salida (sobrescribe / añade)
- `2>` / `2>>` redirige los errores
- `/dev/null` — fichero especial que descarta todo lo que se le escribe

```
$ echo "primera ejecución" > log.txt      # log.txt ahora solo tiene esta línea
$ echo "segunda ejecución" >> log.txt      # añadida, la primera línea sigue ahí
$ ls /nope 2> /dev/null                  # el error "No such file" se descarta en silencio
```

Por qué son tres canales separados, y no solo uno. El propio programa no sabe ni le importa dónde acaba su salida — solo escribe en "su" salida estándar (descriptor 1) y en "su" salida de error (descriptor 2), y es la shell quien decide dónde va cada uno *antes* de que el programa arranque siquiera. Mantener errores y salida normal en canales separados es lo que permite hacer esto:

Pipes y filtros

- `|` conecta la salida estándar de una orden con la entrada de la siguiente — encadena cuantas quieras
- Un **filtro** lee de su entrada estándar, escribe en su salida — la filosofía Unix: programas pequeños combinables
- `sort`, `cut`, `grep`, `head / tail`, `uniq`, `wc`

```
$ ls -l | grep '^d' | wc -l # ¿cuántos subdirectorios hay aquí?
```


Scripting básico (1/2)

```
#!/bin/bash
# shebang: le dice al sistema qué intérprete usar

echo "Hola, $1"      # $1 es el primer argumento
```

- `chmod +x script.sh` para hacerlo ejecutable
- `./script.sh argumento` para ejecutarlo — el `./` hace falta porque el directorio actual no está normalmente en `$PATH`

Scripting básico (2/2)

```
#!/bin/bash
for f in *.txt; do
    if [ -s "$f" ]; then      # -s: el fichero existe y no está vacío
        wc -l "$f"
    fi
done
```

Bucles `for` / `while` , condicionales `if` , `$1` / `$@` / `$#` para argumentos — suficiente para automatizar una tarea repetitiva real.

Checkpoint: fin de S03 (2/2)



Demo en vivo

- Construir en vivo una tubería larga, un filtro cada vez: `cat fichero | grep patrón | sort | uniq -c`
`| sort -rn`
- `top` mientras algo pesado corre en otra terminal (`yes > /dev/null &`), luego `kill` — verlo desaparecer de la lista
- Convertir una orden tecleada a mano en la sesión anterior en un script `.sh` con un bucle

Ejercicios para practicar en casa

- E1 — Mostrar la fecha actual con el formato `31-12-2019`
- E2 — Copiar `/etc/passwd` al directorio actual, una vez con ruta absoluta y otra con ruta relativa
- E3 — Guardar el uso de disco actual en un fichero cuyo nombre incluya la fecha/hora exactas
- E4 — Contar cuántas imágenes hay en `https://www.uah.es`
- E5 (Crear) — Diseñar tu propio script pequeño que automatice una tarea repetitiva

Cierre

- Siguiente tema: `03-os-structure` — modo kernel vs. usuario, llamadas al sistema
- Sigue practicando: `man bash`, `vimtutor`
- Referencia: *The Linux Command Line* — http://linuxcommand.org/lc3_learning_the_shell.php